

PE6201 · Class 5 · Capsule 2 — the cost-to-serve calculator

What this does. Turns an architecture and a success rate into the only cost number a sponsor can act on: **cost per successful task** — plus the **break-even success rate** and a **sensitivity table**.

It costs nothing to run. Prices are *data*, not API calls. Section 8 is the only part that touches a model, and it is optional.

Three rules this notebook enforces, because all three are where the calculation goes wrong:

1. **One currency.** Every figure below is US dollars. Mixing a local-currency wage into US-dollar token prices is the single most common error in this arithmetic.
2. **An agent's input is a SUM over turns, not a multiplication.** The conversation is re-sent every turn.
3. **Never report a point estimate.** The success rate is the term you are least sure of and the one the answer is most sensitive to — so always print the break-even and the sensitivity table beside it.

Sections 1–5 reproduce **Pre-read 3's worked example exactly**, so you can check the notebook against the brief. Section 6 is your exercise on a scenario you have not seen. Section 7 is your own use case.

Section 1 · CONFIG — every price lives here and nowhere else

Change a number in this cell and the whole model re-costs. Nothing is hardcoded further down.

Each price carries the date it was checked, because these move: **Gemini 3.7 Flash launched 13 Aug 2026 on a 50% introductory discount that expires 31 Dec 2026**. A hardcoded price is already wrong.

```
In [1]: # — PRICES · US dollars per 1,000,000 tokens · checked 2026-08-28 against vendor pricing pages —
PRICES = {
    # name                input    output    cached_input (≈0.1× input where the vendor supports it)
    "cheap": {"in": 0.10, "out": 0.40, "cached_in": 0.01}, # e.g. Gemini 2.5 Flash-Lite tier
    "frontier": {"in": 5.00, "out": 25.00, "cached_in": 0.50}, # e.g. Claude Opus tier
}
```

```
# — WHAT A FAILURE COSTS · US dollars per escalation —
#   A failure is not free just because no human is named in the design.
LABOUR = {
    "support_agent": {"usd_per_hour": 45.0, "minutes_per_escalation": 8.0},
    "finance_clerk": {"usd_per_hour": 40.0, "minutes_per_escalation": 0.75},
}

def escalation_cost(role):
    "Cost of one human fallback, in US dollars."
    r = LABOUR[role]
    return r["usd_per_hour"] * r["minutes_per_escalation"] / 60.0

print("support escalation   US$%.2f" % escalation_cost("support_agent"))
print("clerk escalation     US$%.2f" % escalation_cost("finance_clerk"))
```

```
support escalation   US$6.00
clerk escalation     US$0.50
```

Section 2 · The three-layer model

Layer 1 · per-task variable — fresh input + cached input + output + retrieval + tool fees. **Layer 2 · per-task expected fallback** — $(1 - \text{success rate}) \times \text{cost of handling a failure}$. Usually the largest layer, and the one almost every cost model omits. **Layer 3 · fixed monthly** — vector store, infrastructure, eval runs, monitoring, maintenance.

Keep them apart. They behave differently as volume grows: layer 1 and 2 are linear and never amortise, layer 3 gets cheaper per task forever.

```
In [2]: def agent_input_tokens(base, growth, turns):
        """Input tokens an agent sends across a whole run.

        The API is STATELESS. Nothing is remembered between calls, so the whole
        conversation so far is re-sent as input on every turn. Turn t sends
        base + growth*(t-1), and summed over T turns:

            base*T + growth * T*(T-1)/2          <- quadratic in turns, not linear

        Keep the two terms in proportion. base*T is LINEAR (the fixed prompt re-sent
        every turn); only the second term is quadratic. The quadratic one overtakes
```

the linear one at turns $> 2 \times \text{base} / \text{growth} + 1$ -- about 15 turns for 5,500/800.
At 6 turns it is only ~27% of the bill, so on short loops shrink `base` first.

`growth` is everything one completed turn adds to the conversation:

growth = the model's own reply (its thought + chosen action)
 + the observation the tool returned

Both halves are re-sent as INPUT on every later turn. So the model's reply is paid for twice over: ONCE at the OUTPUT price, on the turn it is written, and then at the INPUT price on every turn after that. There is nothing further to add for replies -- they are already inside `growth`.

```
return base * turns + growth * turns * (turns - 1) // 2
```

```
def variable_cost(tier, fresh_in=0, cached_in=0, out=0, retrieval_usd=0.0, tool_usd=0.0):
    "LAYER 1 - per-task variable cost, US dollars."
    p = PRICES[tier]
    assert fresh_in >= 0 and cached_in >= 0 and out >= 0, "token counts cannot be negative"
    return (fresh_in / 1e6 * p["in"]
            + cached_in / 1e6 * p["cached_in"]
            + out / 1e6 * p["out"]
            + retrieval_usd + tool_usd)

def cost_per_successful_task(var_usd, success_rate, failure_usd):
    "LAYER 1 + LAYER 2. The number to quote."
    assert 0.0 <= success_rate <= 1.0, "success rate is a probability, not a percentage"
    assert failure_usd >= 0, "a failure cannot cost less than nothing"
    return var_usd + (1.0 - success_rate) * failure_usd

def monthly(var_usd, success_rate, failure_usd, volume, fixed_monthly_usd=0.0):
    "Everything, for a month."
    return cost_per_successful_task(var_usd, success_rate, failure_usd) * volume + fixed_monthly_usd

def break_even_success_rate(cheap_var, dear_total, failure_usd):
    """The success rate the CHEAP option needs to match the expensive one.
```

```

cheap_var + (1-p) * failure = dear_total -> p = 1 - (dear_total - cheap_var)/failure
,,,,,
p = 1.0 - (dear_total - cheap_var) / failure_usd
return max(0.0, min(1.0, p))

```

Section 3 · Pre-read 3's worked example, reproduced

A support-triage assistant, **50,000 tickets a month**. Four architectures, token cost only for now.

Check these against the brief — they are the same numbers on purpose.

```

In [3]: VOLUME = 50_000

v1 = variable_cost("cheap",    fresh_in=1_500,  out=400)
v2 = variable_cost("cheap",    fresh_in=5_500,  out=400)           # +4,000 tokens of retrieved context

# v3/v4: a six-turn agent. Each completed turn adds ~800 tokens to the conversation:
# ~400 of the model's own reply (thought + action) and ~400 of tool observation.
TURNS, BASE      = 6, 5_500
REPLY, OBSERVATION = 400, 400
GROWTH           = REPLY + OBSERVATION           # = 800
agent_in = agent_input_tokens(BASE, GROWTH, TURNS)
agent_out = REPLY * TURNS

v3 = variable_cost("cheap",    fresh_in=agent_in, out=agent_out)
v4 = variable_cost("frontier", fresh_in=agent_in, out=agent_out)

rows = [("v1 · one call", v1), ("v2 · + retrieval", v2),
        ("v3 · 6-turn agent, cheap", v3), ("v4 · same agent, frontier", v4)]
print(f"{'architecture':<30}{'per task':>12}{'per month':>14}")
for name, c in rows:
    print(f"{'name':<30}{c:>12.5f}{c*VOLUME:>14,.2f}")

print(f"\nagent input: {BASE:,}x{TURNS} base + {GROWTH}x{TURNS*(TURNS-1)//2} carried forward"
      f" = {agent_in:,} tokens")
print(f"  of which the model's own replies re-sent as input: "
      f"{REPLY*TURNS*(TURNS-1)//2:,} tokens")
print(f"  the same replies also cost {agent_out:,} tokens at the OUTPUT price, once each.")

```

```
print(f"\nv4 is {v4/v3:.0f}x v3 and {round(v4/v1, -1):.0f}x v1, for one feature.")

assert round(v1, 5) == 0.00031 and round(v2, 5) == 0.00071
assert round(v3, 5) == 0.00546 and round(v4, 3) == 0.285
print("\n✓ matches Pre-read 3")
```

architecture	per task	per month
v1 · one call	0.00031	15.50
v2 · + retrieval	0.00071	35.50
v3 · 6-turn agent, cheap	0.00546	273.00
v4 · same agent, frontier	0.28500	14,250.00

agent input: 5,500x6 base + 800x15 carried forward = 45,000 tokens
 of which the model's own replies re-sent as input: 6,000 tokens
 the same replies also cost 2,400 tokens at the OUTPUT price, once each.

v4 is 52x v3 and 920x v1, for one feature.

✓ matches Pre-read 3

Section 4 · Add the success rate — and watch the ranking invert

v3 resolves **55%** of tickets unaided, v4 resolves **80%**. Every unresolved ticket falls to a human: eight minutes at a loaded **US 45/hour = 6.00**.

On cost per *token*, v4 is 52x worse. Now ask the question that matters.

```
In [4]: FAILURE = escalation_cost("support_agent")          # US$6.00

t3 = cost_per_successful_task(v3, 0.55, FAILURE)
t4 = cost_per_successful_task(v4, 0.80, FAILURE)

print(f"v3 tokens {v3:.5f} + fallback {0.45*FAILURE:.2f} = ${t3:.3f} per RESOLVED ticket")
print(f"v4 tokens {v4:.5f} + fallback {0.20*FAILURE:.2f} = ${t4:.3f} per RESOLVED ticket (the slide rounds to
print(f"\nthe 52x-more-expensive architecture is {1-t4/t3:.0%} CHEAPER per resolved ticket")
print(f"on {VOLUME:,} tickets/month that is ${ (t3-t4)*VOLUME:,.0f} a month, "
      f"${ (t3-t4)*VOLUME*12:,.0f} a year")

# where does the money actually go?
```

```

for label, tok, tot in (("v3", v3, t3), ("v4", v4, t4)):
    print(f"\n{label}: the human fallback is {(tot-tok)/tot:.1%} of cost per resolved ticket")

p = break_even_success_rate(v3, t4, FAILURE)
print(f"\nBREAK-EVEN: v3 wins once its resolution rate reaches {p:.1%}")
assert 0.75 <= p <= 0.76
print("✓ matches Pre-read 3 (75.3%)")

```

v3 tokens 0.00546 + fallback 2.70 = \$2.705 per RESOLVED ticket
v4 tokens 0.28500 + fallback 1.20 = \$1.485 per RESOLVED ticket (the slide rounds to \$1.49)

the 52x-more-expensive architecture is 45% CHEAPER per resolved ticket
on 50,000 tickets/month that is \$61,023 a month, \$732,276 a year

v3: the human fallback is 99.8% of cost per resolved ticket

v4: the human fallback is 80.8% of cost per resolved ticket

BREAK-EVEN: v3 wins once its resolution rate reaches 75.3%
✓ matches Pre-read 3 (75.3%)

Section 5 · Never ship a point estimate

The success rate is an estimate. So is the other one. Print the table, not the number.

```

In [5]: def sensitivity(var_usd, failure_usd, centre, spread=0.10, step=0.05, label=""):
    print(f"{label} cost per successful task, success rate ±{spread:.0%}")
    r = centre - spread
    while r <= centre + spread + 1e-9:
        mark = " <- estimate" if abs(r - centre) < 1e-9 else ""
        print(f" {r:6.0%} ${cost_per_successful_task(var_usd, r, failure_usd):8.3f}{mark}")
        r += step

sensitivity(v3, FAILURE, 0.55, label="v3 .")
print()
sensitivity(v4, FAILURE, 0.80, label="v4 .")

print("\nRead it this way: v4 stays cheaper across the whole plausible range,")

```

```
print("so the decision is robust. That is worth saying out loud to a sponsor -")
print("a robust answer and a knife-edge answer deserve different amounts of confidence.")
```

```
v3 · cost per successful task, success rate ±10%
    45% $ 3.305
    50% $ 3.005
    55% $ 2.705 <- estimate
    60% $ 2.405
    65% $ 2.105
```

```
v4 · cost per successful task, success rate ±10%
    70% $ 2.085
    75% $ 1.785
    80% $ 1.485 <- estimate
    85% $ 1.185
    90% $ 0.885
```

Read it this way: v4 stays cheaper across the whole plausible range, so the decision is robust. That is worth saying out loud to a sponsor – a robust answer and a knife-edge answer deserve different amounts of confidence.

Section 6 · YOUR TURN — a scenario you have not seen

Supplier-invoice matching at a regional distributor. 12,000 invoices a month. Extract the fields, match against the purchase order, flag discrepancies.

A failure is cheap here: a finance clerk glances at the flag — **45 seconds at US\$40/hour**.

	Architecture	Tokens	Resolves
A	cheap model, one call + retrieval	4,000 in · 250 out	75%
B	frontier model, 4-turn agent	19,600 in · 1,000 out	92%

Fill in the four `### REPLACE ME` lines. Then answer, in this order: **(1)** cost per successful task for each · **(2)** which wins, and by how much a year · **(3)** the break-even · **(4)** does your answer go the same way as the worked example, and *why not*?

```

In [6]: INVOICE_VOLUME = 12_000
FAILURE_B = escalation_cost("finance_clerk")      # a much cheaper mistake

A = variable_cost("cheap",    fresh_in=0, out=0)   ### REPLACE ME – 4,000 in, 250 out
B = variable_cost("frontier", fresh_in=0, out=0)   ### REPLACE ME – 19,600 in, 1,000 out

A_success = 0.0                                  ### REPLACE ME
B_success = 0.0                                  ### REPLACE ME

# --- nothing below here needs changing ---
if A > 0 and B > 0 and A_success > 0 and B_success > 0:
    ta = cost_per_successful_task(A, A_success, FAILURE_B)
    tb = cost_per_successful_task(B, B_success, FAILURE_B)
    winner, loser = ("A", "B") if ta < tb else ("B", "A")
    print(f"A  ${ta:.4f} per successful task")
    print(f"B  ${tb:.4f} per successful task")
    print(f"\n{winner} wins by {abs(1 - min(ta,tb)/max(ta,tb)):.0%}  "
          f"– ${abs(ta-tb)*INVOICE_VOLUME:,.0f}/month, ${abs(ta-tb)*INVOICE_VOLUME*12:,.0f}/year")
    print(f"break-even for A: {break_even_success_rate(A, tb, FAILURE_B):.1%}")
    sensitivity(A, FAILURE_B, A_success, label="\nA .")
else:
    print("Fill in the four REPLACE ME lines above.")

```

Fill in the four REPLACE ME lines above.

Check yourself

Run this once you have an answer of your own — not before.

```

In [7]: def check():
    a = variable_cost("cheap",    fresh_in=4_000, out=250)
    b = variable_cost("frontier", fresh_in=19_600, out=1_000)
    ta = cost_per_successful_task(a, 0.75, FAILURE_B)
    tb = cost_per_successful_task(b, 0.92, FAILURE_B)
    print(f"A  tokens ${a:.4f}  total ${ta:.4f}")
    print(f"B  tokens ${b:.4f}  total ${tb:.4f}")
    print(f"A wins by {1-ta/tb:.0%} – ${(tb-ta)*INVOICE_VOLUME*12:,.0f} a year")
    print(f"break-even {break_even_success_rate(a, tb, FAILURE_B):.1%}")
    print("\nWHY THE OPPOSITE ANSWER: a support escalation costs $6.00; a clerk")

```

```

print("glancing at a flagged invoice costs $0.50 – twelve times less. So B's")
print("17-point resolution advantage buys twelve times less. The fallback cost")
print("decides, not the model. A rule of thumb would have got this backwards.")

```

```
check()
```

A tokens \$0.0005 total \$0.1255
 B tokens \$0.1230 total \$0.1630
 A wins by 23% – \$5,400 a year
 break-even 67.5%

WHY THE OPPOSITE ANSWER: a support escalation costs \$6.00; a clerk
 glancing at a flagged invoice costs \$0.50 – twelve times less. So B's
 17-point resolution advantage buys twelve times less. The fallback cost
 decides, not the model. A rule of thumb would have got this backwards.

Section 7 · Your own use case

Now the one you actually care about — for **A2**, for your project, or for work. If you cannot fill in the success rate, that is the finding: **you do not yet know what it costs.**

```

In [8]: MY = dict(
    volume           = 0,          ### REPLACE ME – tasks per month
    tier              = "cheap",    ### "cheap" or "frontier"
    fresh_in         = 0,          ### REPLACE ME
    cached_in       = 0,          # tokens you can serve from cache at ~0.1x
    out              = 0,          ### REPLACE ME
    retrieval_usd    = 0.0,        # per task, if any
    success_rate     = 0.0,        ### REPLACE ME – be honest, then test it in Section 5
    failure_usd      = 0.0,        ### REPLACE ME – what handling one failure costs
    fixed_monthly_usd = 0.0,       # vector store, monitoring, the engineer who maintains it
)

if MY["volume"]:
    var = variable_cost(MY["tier"], MY["fresh_in"], MY["cached_in"], MY["out"], MY["retrieval_usd"])
    per = cost_per_successful_task(var, MY["success_rate"], MY["failure_usd"])
    tot = monthly(var, MY["success_rate"], MY["failure_usd"], MY["volume"], MY["fixed_monthly_usd"])
    print(f"variable per task      ${var:.5f}")
    print(f"cost per SUCCESSFUL task ${per:.4f}")

```

```

print(f"monthly, all in          ${tot:,.2f}")
print(f"the fallback is {(per-var)/per:.0%} of your cost per successful task")
sensitivity(var, MY["failure_usd"], MY["success_rate"], label="\nsensitivity .")
else:
    print("Fill in MY above.")

```

Fill in MY above.

Section 8 · OPTIONAL — measure your own token counts

Everything above used token counts I gave you. For your own use case you need real ones. This is the only cell that calls a model, it uses **your OpenRouter key**, and it costs a fraction of a cent.

Skip it if you are offline — the calculator does not depend on it.

```

In [9]: BACKEND = "scripted"          # "scripted" = no network. Set to "live" to measure for real.
MODEL     = "openai/gpt-4o-mini"
BASE_URL  = "https://openrouter.ai/api/v1"

def measure(prompt, backend=BACKEND):
    """Return (input_tokens, output_tokens) for one call. One function knows a vendor exists."""
    if backend == "scripted":
        return (len(prompt) // 4, 180)          # ~4 chars per token, plausible reply
    import os, json, urllib.request
    key = os.environ.get("OPENROUTER_API_KEY", "")
    assert key, "set OPENROUTER_API_KEY first"
    req = urllib.request.Request(
        BASE_URL + "/chat/completions",
        data=json.dumps({"model": MODEL, "messages": [{"role": "user", "content": prompt}]}).encode(),
        headers={"Authorization": "Bearer " + key, "Content-Type": "application/json"})
    u = json.load(urllib.request.urlopen(req, timeout=60))["usage"]
    return (u["prompt_tokens"], u["completion_tokens"])

try:
    tin, tout = measure("Summarise this support ticket in one line: printer offline since Tuesday, "
                        "reboot did not help, user is in the Singapore office.")
    print(f"input {tin} tokens · output {tout} tokens")
    print(f"cost at the cheap tier: ${variable_cost('cheap', fresh_in=tin, out=tout):.6f} per call")
    print(f"cost at the frontier : ${variable_cost('frontier', fresh_in=tin, out=tout):.6f} per call")

```

```
except Exception as e:  
    print("skipped:", e)
```

input 31 tokens · output 180 tokens
cost at the cheap tier: \$0.000075 per call
cost at the frontier : \$0.004655 per call

What to take away

1. **Cost per successful task**, never cost per call. The two rank options in opposite orders.
2. **The fallback term is usually the largest** — 81% to 99.8% of cost in the worked example.
3. **Report the break-even and the sensitivity.** A robust answer and a knife-edge answer deserve different amounts of confidence, and only the table shows which you have.
4. **Prices are data.** One config block, dated. At least one price on this page will be wrong by January.

Built for PE6201 Class 5. Numbers reproduce Pre-read 3 exactly — if the brief changes, change both.